

# SWK Klausur WS16/17

Bei Dr. Boris Düdder

24.02.2017 (Ersttermin),  
60 Minuten (Zeit hat gerade so gereicht)

## 1 OCL (15 Punkte)

Es waren 5 verschiedene Aussagen zu einem gegebenem Klassendiagramm in OCL zu modellieren. 3 davon waren Invarianten, die letzten 2 waren Aussagen zu Funktionen aus den Klassen.

## 2 Petrinetze - Modellierung (15 Punkte)

Zu Modellieren war eine Ampel mit folgenden Eigenschaften:

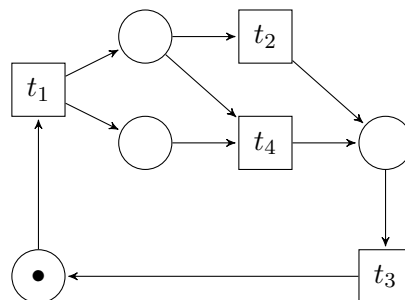
- Ist die Ampel rot, so kann sie im nächsten Schritt rot und gelb werden.
- Ist sie rot und gelb, so kann sie im nächsten Schritt grün werden.
- Ist sie grün, so kann sie im nächsten Schritt gelb werden.
- Ist sie gelb, so kann sie im nächsten Schritt rot werden.

Die Modellierung sollte ausschließlich mit drei Zuständen (grün, gelb, rot) erfolgen.

Außerdem war ein Petrinetz gegeben, welches man durch Einzeichnen von Verbindungen zu einem lebendigen Netz machen sollte.

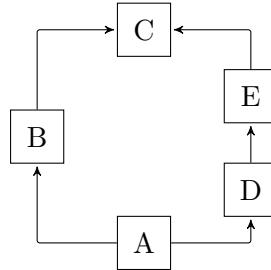
## 3 Petrinetze - Eigenschaften (10 Punkte)

Gegeben war das folgende Petrinetz, zu dem ein Erreichbarkeitsgraph aufzustellen und die Eigenschaften *tot*, *lebendig*, *deadlockfrei*, *k-beschränkt* und *sicher* zu bestimmen waren:



## 4 CK-Metriken (5 Punkte)

Gegeben war ein dem folgenden Klassendiagramm ähnliches Bild:



Zu den Klassen sollten die Metriken DIT, NOC und WMC aufgestellt werden.

## 5 Zustandsbasiertes Testen (10 Punkte)

Ein Übergangsbaum sollte gezeichnet werden, dazu sollten entsprechend die beiden Konformanz- und Robustheitstests gemacht werden (Zustandsüberdeckung und Zustandsübergangsüberdeckung).

## 6 Kontrollflussbasiertes Testen (10 Punkte)

Entscheidungsüberdeckung, Anweisungsüberdeckung etc...

## 7 Algebraische Spezifikation (30 Punkte)

Es waren folgende Funktionen gegeben, welche auf einer Listen-Datenstruktur definiert war und für die eine vollständige algebraische Spezifikation angegeben werden sollte:

- `create`: Erzeugt eine leere Liste.
- `add`: Fügt ein Element hinten an zu einer Liste hinzu.
- `remove`: Entfernt das letzte Element einer Liste.
- `comp`: Hängt den Inhalt einer Liste an eine andere.
- `head`: Gibt das oberste Element einer Liste zurück.
- `length`: Gibt die Länger der Liste zurück.

Als zweite Unteraufgabe war die Liste  $[1, 2, 3, 4]$  gegeben. (Als Hinweis war angegeben, dass dies eine Kurzschreibweise war von:  $\text{add}(\text{add}(\text{add}(\text{add}(\text{create}(), 1), 2), 3), 4)$ ). Man sollte die Anfrage  $\text{tail}([1, 2, 3, 4])$  lösen ("tail" war eine neue Funktion, welche immer eine Liste ohne das vorderste Element zurück geben sollte) mittels folgender Axiome:

- $\text{tail}(\text{add}(\text{add}(l, e_1), e_2)) = \text{add}(\text{tail}(\text{add}(l, e_1)), e_2)$
- $\text{tail}(\text{add}(\text{create}(), e)) = e$